

Министерство науки и высшего образования Российской Федерации
Муромский институт (филиал)
федерального государственного бюджетного образовательного учреждения высшего образования
**«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»**
(МИ ВлГУ)

Факультет информационных технологий радиоэлектроники
Кафедра информационных систем

КУРСОВАЯ РАБОТА

по курсу Прикладная разработка на Java
на тему: Создание адаптивной платформы локального медиаконтента с
динамической сменой хранилища

Руководитель

(оценка)

Метелкин А. С.
(фамилия, инициалы)

(подпись) (дата)

Члены комиссии

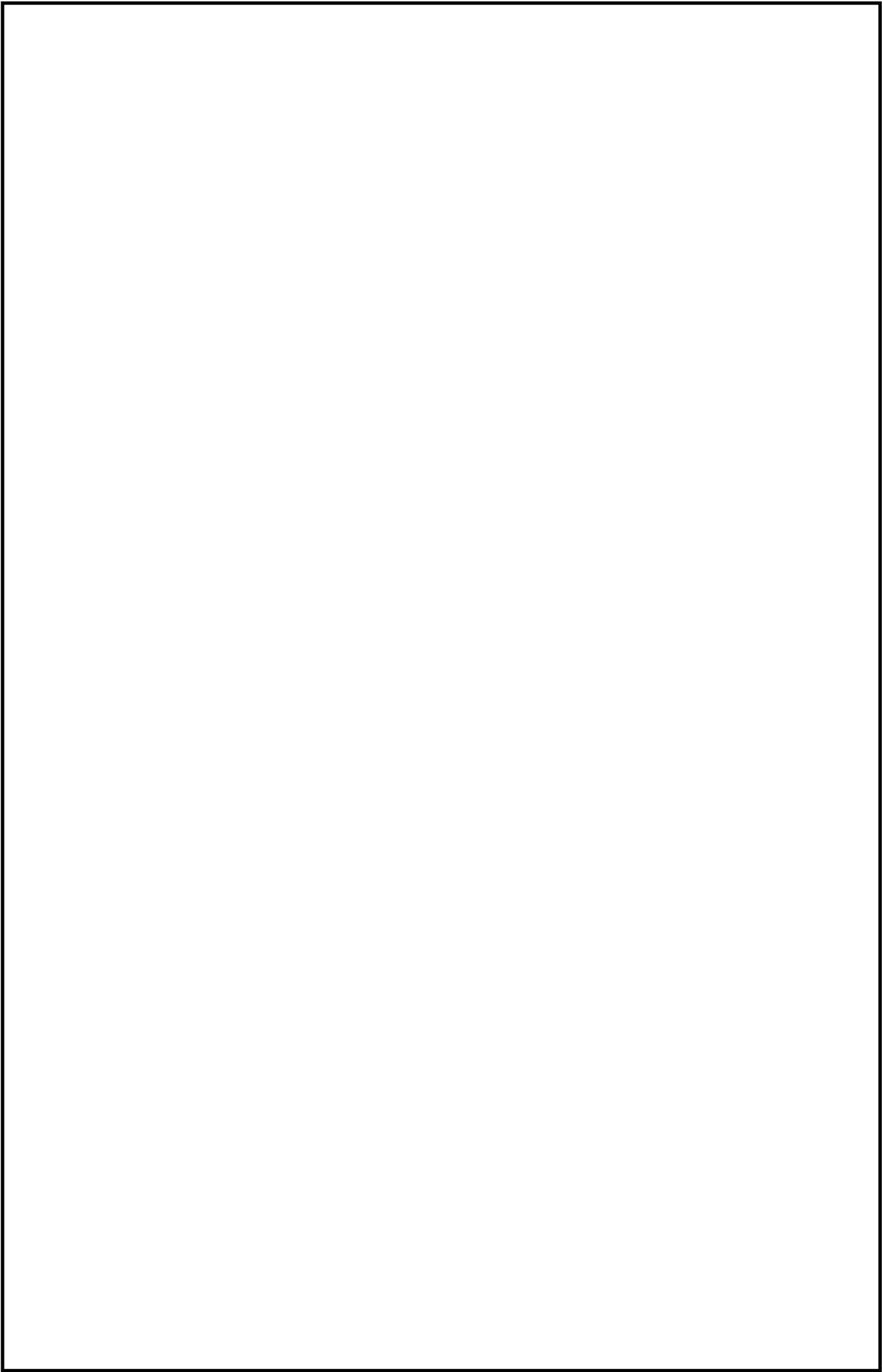
Студент ИС-123
(группа)

(подпись)(Ф.И.О.)

Медведев С. В.
(фамилия, инициалы)

(подпись)(Ф.И.О.)

(подпись)(дата)

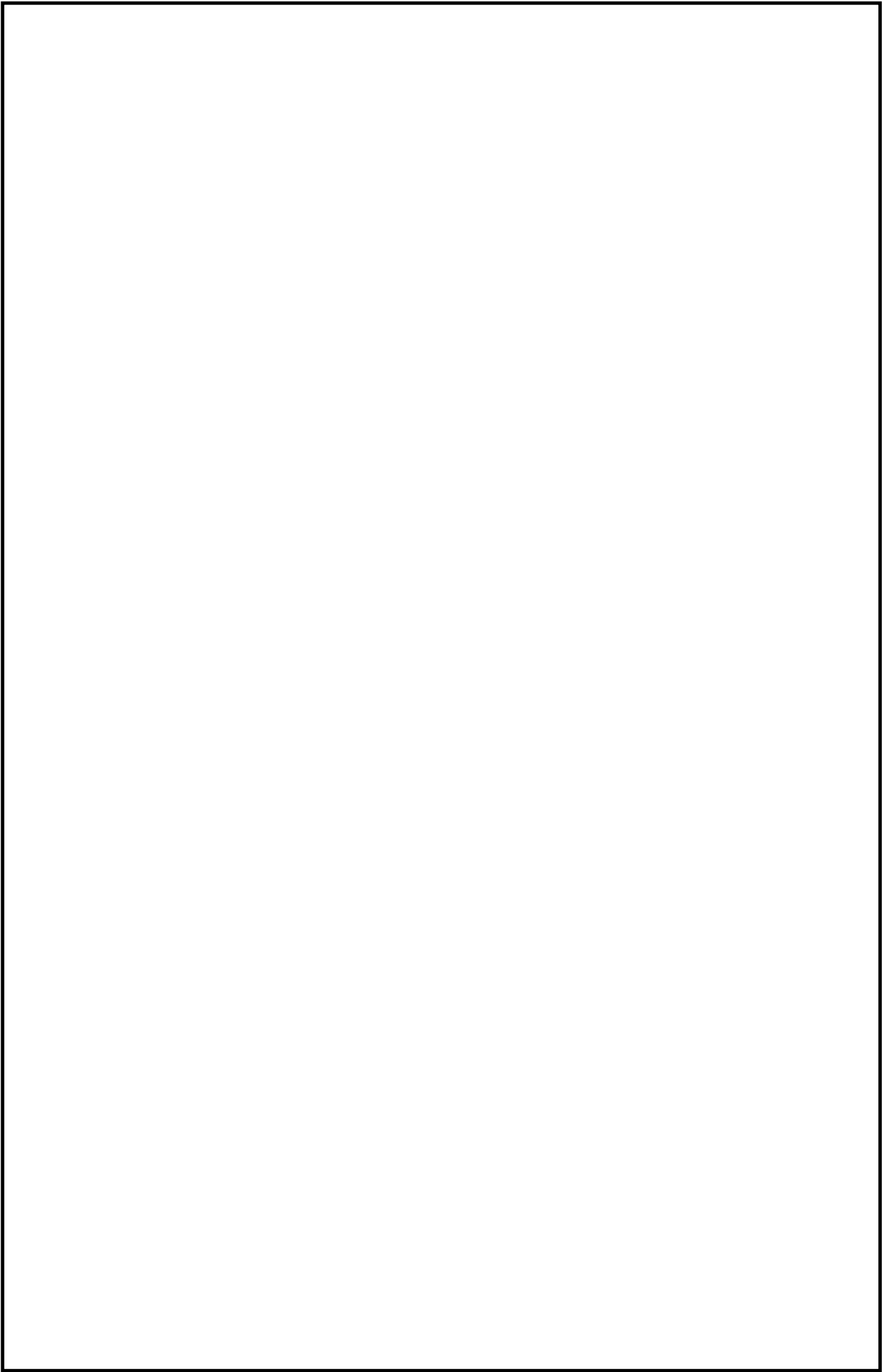


В курсовом проекте отражена разработка веб-приложения — домашней платформы для каталогизации и воспроизведения локального медиаконтента (фильмов и сериалов) на языке Java 21 с использованием фреймворка Spring Boot 3.4. Ключевой особенностью проекта является динамическая смена хранилища в рамках пользовательской сессии: система поддерживает два режима работы — реляционная СУБД Firebird (через Spring Data JPA / Hibernate) и файловое хранилище в формате JSON (через Jackson Databind). Переключение между источниками данных реализовано без перезапуска приложения и без изменения бизнес-логики.

В ходе работы спроектирована гексагональная (Port–Adapter) архитектура с разделением на слои Controller / Service / Port / Adapter / Repository, что обеспечивает слабую связанность модулей и лёгкую расширяемость. Реализована многоуровневая система безопасности с ролевой моделью (ADMIN, moderator, user) на базе Spring Security и алгоритма хеширования паролей BCrypt. Пользовательский интерфейс построен на серверном шаблонизаторе Thymeleaf с использованием встроенного HTML5-плеера для воспроизведения видеофайлов. Дополнительно реализован механизм рекомендаций на основе паттерна Observer (уведомления о релизах) и генератора настроения MoodAnalyzer.

Корректность работы ключевых модулей подтверждена модульными тестами (JUnit 5 + Mockito + AssertJ). Разработанный сервис представляет собой готовый инструмент для организации локального медиа-каталога и может быть использован в образовательных и домашних сценариях.

Табл. 3. Ил. 15. Библ. 5.



СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
1 Анализ технического задания.....	6
1.1 Цель и задачи разработки.....	6
1.2 Сравнительный анализ существующих решений.....	6
1.3 Выбор и обоснование технологического стека.....	8
1.4 Архитектура системы.....	9
1.5 Управление медиаконтентом.....	11
2 Проектирование программы.....	12
2.1 Проектирование архитектуры и структуры данных системы.....	12
2.2 Проектирование информационной модели и базы данных.....	13
3 Разработка приложения.....	15
3.1 Архитектурные особенности реализации.....	15
3.2 Реализация информационной модели и базы данных.....	15
3.3 Реализация системы адаптеров хранилища (Port–Adapter).....	16
3.4 Реализация управления жизненным циклом медиаконтента.....	17
3.5 Реализация интерфейса просмотра и воспроизведения.....	18
3.6 Реализация модуля рекомендаций и уведомлений.....	19
4 Тестирование.....	20
ЗАКЛЮЧЕНИЕ.....	35
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	37

					МИВУ.09.03.02-09.000										ПЗ				
Изм.	Лист	№ докум.	Подпись	Дата	Курсовой проект по теме: «Создание адаптивной платформы локального медиаконтента с динамической сменой хранилища»										Лит.	Лист	Листов		
Разраб.		Медведев С. В.													У			4	37
Провер.		Метелкин А.С.													ИС-123				
Реценз.																			
Н. Контр.																			
Утверд.																			

функциональная страница

					МИВУ.09.03.02-09.000			ПЗ	
Изм.	Лист	№ докум.	Подпись	Дата					
Разраб.		Медведев С. В.			Курсовой проект по теме: «Создание адаптивной платформы локального медиаконтента с динамической сменой хранилища»			Лит.	Лист
Провер.		Метелкин А.С.							
Реценз.								4	37
Н. Контр.								ИС-123	
Утверд.									

ВВЕДЕНИЕ

С развитием цифровых технологий и повсеместным распространением локальных медиатек всё более востребованными становятся инструменты для каталогизации, поиска и воспроизведения аудио- и видеофайлов в пределах одной домашней сети. Современные веб-технологии позволяют создавать интерактивные сервисы, которые делают процесс работы с медиаконтентом удобным и эстетически привлекательным [1].

Тема курсового проекта — создание адаптивной платформы локального медиаконтента с динамической сменой хранилища. Объектом является процесс организации, хранения и воспроизведения локальной видеотеки.

Цель курсового проекта — спроектировать и разработать многофункциональное веб-приложение для управления каталогом фильмов и сериалов с возможностью воспроизведения видео через встроенный HTML5-плеер и с поддержкой двух независимых механизмов хранения данных (СУБД и JSON-файлы).

Исходя из поставленной цели, определены следующие задачи:

- изучение литературных источников по теме;
- проектирование базы данных;
- реализация серверной части приложения (backend) с использованием паттернов проектирования (Port–Adapter, Strategy, Observer);
- создание пользовательского интерфейса (frontend) на базе Thymeleaf;
- реализация системы ролей и безопасности;
- тестирование и отладка разработанного сервиса.

Результатом работы является готовое веб-приложение, предоставляющее каталог, плеер, административную панель и возможность переключения между режимами хранения данных.

1 Анализ технического задания

1.1 Цель и задачи разработки

Современная медиасреда характеризуется экспоненциальным ростом объёмов цифрового контента. Традиционные файловые менеджеры не предоставляют удобной каталогизации, контекстного поиска или персонализированных рекомендаций. Особую актуальность задача приобретает в контексте цифрового суверенитета: большинство популярных медиасерверов являются проприетарными решениями, зависимыми от внешних облачных сервисов.

Дополнительным фактором актуальности является потребность в гибких механизмах хранения. В реальных условиях развёртывание полноценной СУБД не всегда целесообразно (маломощное оборудование, демонстрационные сценарии), но при этом требуется сохранить возможность миграции на промышленное хранилище без переписывания бизнес-логики. Данная задача напрямую соотносится с принципами гексагональной архитектуры (Ports & Adapters) [3].

Для достижения поставленной цели необходимо выполнить декомпозицию и решить следующие задачи:

- провести анализ существующих решений (аналогов) на рынке;
- выбрать оптимальный стек технологий и инструментов для разработки;
- спроектировать структуру базы данных для хранения форм, вопросов и ответов;
- разработать серверную часть (backend) с использованием паттернов проектирования и реализовать бизнес-логику;
- создать пользовательский интерфейс (frontend) для трех основных ролей (администратор, модератор, пользователь);
- реализовать функционал динамического переключения хранилища;

					МИВУ.09.03.02-09.000	ПЗ	Лист
							6
Изм.	Лист	№ докум.	Подпись	Дата			

- провести модульное (Unit Testing) и ручное тестирование готового приложения.

1.2 Сравнительный анализ существующих решений

В ходе исследования были рассмотрены популярные решения для управления медиатеками: Plex Media Server, Jellyfin и Emby. Каждый из них обладает уникальными характеристиками, определяющими его целевую аудиторию и сценарии использования.

Plex Media Server — коммерческий продукт с закрытым исходным кодом, предоставляющий широкий спектр функций. Однако его ключевым недостатком является обязательная привязка к облачным серверам Plex для аутентификации. Это создаёт зависимость от стороннего сервиса и ограничивает использование в автономных средах.

Jellyfin — полностью бесплатный форк проекта Emby с открытым исходным кодом. Он не требует подключения к внешним серверам. В качестве хранилища метаданных используется встроенная СУБД SQLite, что делает его легковесным, но ограничивает возможности масштабирования. Архитектура не предполагает динамической смены хранилища.

Emby — проприетарный продукт, сочетающий черты Plex и Jellyfin. Хранение данных также реализовано через SQLite, без возможности переключения на альтернативные источники без сложных модификаций.

Анализ аналогов и обоснование выбора технологий показаны в таблице 1.

Таблица 1 — Анализ аналогов и разрабатываемого решения

Критерий сравнения	Plex Media Server	Jellyfin	Emby	Разрабатываемое приложение
Среда развертывания	Облако + локально	Облако (SaaS)	Локальный сервер	Локальный сервер
Хранение данных	Проприетарная БД	SQLite	SQLite	Firebird SQL / JSON
Исходный код	Закрытый	Открытый (GPL-2.0)	Закрытый	Открытый
Стоимость использования	Freemium	Бесплатно	Freemium	Бесплатно

Продолжение таблицы 1 — Анализ аналогов и разрабатываемого решения

Критерий сравнения	Plex Media Server	Jellyfin	Emby	Разрабатываемое приложение
Динамическая смена хранилища	Отсутствует	Отсутствует	Отсутствует	Реализована (DB / JSON)
Автономная работа	Ограничена	Полная	Частичная	Полная
Критерий сравнения	Plex Media Server	Jellyfin	Emby	Разрабатываемое приложение
Интеграция со своей БД	Только через API	Через плагины	Через REST API	Прямая работа (JPA/Jackson)
Транскодирование видео	Есть (в т.ч. аппаратное)	Есть (FFmpeg)	Есть (платно)	Отсутствует (не требуется)
Ролевая модель	2 роли (admin/user)	2 роли	3 роли	3 роли (ADMIN/moderator/user)

Таким образом, на основе проведенного сравнительного анализа можно сделать вывод, что разработка собственного веб-сервиса на стеке Java 21 и Spring Boot 3 является наиболее целесообразной. Это позволит реализовать гибкую систему управления медиаконтентом с возможностью динамического переключения между реляционной БД Firebird SQL и файловым хранилищем JSON, обеспечивая полную независимость от сторонних облачных сервисов [4].

1.3 Выбор и обоснование технологического стека

Для реализации серверной части приложения был выбран язык программирования Java 21 [1] и фреймворк Spring Boot 3.4. Данный выбор обусловлен необходимостью создания масштабируемого и безопасного приложения с использованием современных стандартов разработки [5].

Основными компонентами технологического стека стали:

- Spring Data JPA и Hibernate 6. В отличие от классического подхода с ручным управлением JDBC-соединениями, использование ORM-решения позволило автоматически сопоставлять Java-классы с таблицами в БД Firebird. Настройка Hibernate Community Dialects обеспечивает корректную работу

генерации ID и типов данных BLOB.

- Spring Security. Для защиты пользовательских данных и разграничения прав доступа (ADMIN/moderator/user) была внедрена подсистема безопасности. Она реализует механизмы аутентификации через сессии и обеспечивает надежное хранение паролей с использованием алгоритма хеширования BCrypt.

- Thymeleaf. В качестве движка представлений был выбран серверный шаблонизатор Thymeleaf. Его преимущество заключается в возможности создания «натуральных» шаблонов, которые остаются валидными HTML-файлами, что упростило верстку интерфейса.

- Jackson Databind. Используется для сериализации и десериализации данных в формате JSON при работе с файловым хранилищем, обеспечивая глубокую интеграцию со Spring Boot.

- Lombok. Библиотека была применена для минимизации шаблонного кода (boilerplate). С помощью аннотаций @Data, @NoArgsConstructor удалось значительно сократить объем кода в моделях данных.

1.4 Архитектура системы

Веб-сервисы для управления медиаконтентом описываются законами клиент-серверной архитектуры и принципами проектирования баз данных [2]. Для проекта была выбрана гексагональная архитектура (Ports & Adapters), (рисунок 1) поскольку она наилучшим образом решает ключевую задачу проекта — динамическую смену хранилища данных.

В этой архитектуре бизнес-логика зависит только от абстрактных интерфейсов (портов), а конкретные реализации (адаптеры) подключаются через механизм Dependency Injection.

Порт (Port) — Java-интерфейс, определяющий контракт доступа к данным (например, MoviePort, SeriesPort).

Адаптер (Adapter) — конкретная реализация порта. Для каждого порта существуют две параллельные реализации: DbAdapter и JsonAdapter.

					МИВУ.09.03.02-09.000	ПЗ	Лист
							9
Изм.	Лист	№ докум.	Подпись	Дата			

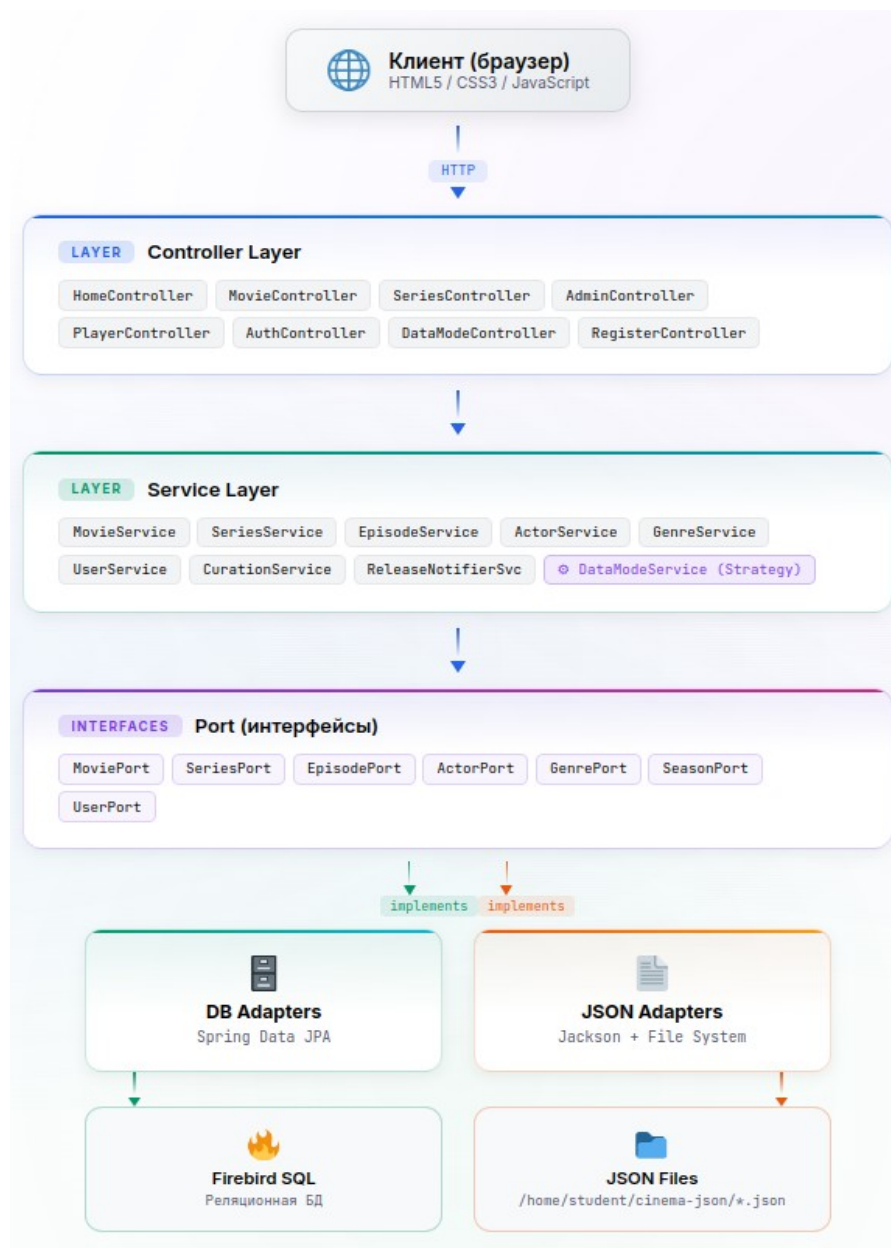


Рисунок 1 – Общая архитектурная схема

Жизненный цикл данных в системе можно описать следующими этапами:

- этап проектирования и создания — администратор или модератор через административную панель добавляет фильмы, сериалы, жанры и актеров;
- этап каталогизации — система индексирует добавленный контент, формирует связи между сущностями и сохраняет метаданные в выбранное хранилище;
- этап потребления — пользователь просматривает каталог и воспроизводит видео через HTML5-плеер;

- этап рекомендаций — система формирует персональные рекомендации на главной странице.

1.5 Управление опросами

Создание, редактирование и удаление медиаконтента являются основными функциями системы. Авторизованные пользователи с ролью ADMIN или moderator имеют доступ к панели управления.

1.5.1 Просмотр и фильтрация медиаконтента

На публичных страницах приложения отображаются списки фильмов, сериалов, жанров и актеров. Для сериалов реализована иерархическая навигация: переход от списка сериалов к списку эпизодов конкретного сезона, а затем к плееру.

1.5.2 Поддерживаемые типы сущностей

В системе реализована поддержка восьми основных типов сущностей:

- Фильм (Movie): базовая сущность с названием, описанием, путем к файлу, годом выпуска и связями с жанрами и актерами.

- Сериал (Series): сущность для многосерийного контента, содержащая метаданные о количестве сезонов и эпизодов.

- Сезон (Season) и Эпизод (Episode): реализуют иерархию сериала со связями ON DELETE CASCADE.

- Справочники: Жанры (Genre), Актеры (Actor), Режиссеры (Director).

- Пользователь (User): сущность системы безопасности с хешированным паролем и ролью.

					МИВУ.09.03.02-09.000	ПЗ	Лист
							11
Изм.	Лист	№ докум.	Подпись	Дата			

2 Проектирование программы

2.1 Проектирование архитектуры и структуры данных системы

Для разработки многофункционального веб-сервиса на начальном этапе была проведена детальная формализация основных задач проекта. Ключевым требованием стала необходимость создания гибкой системы, способной переключаться между реляционной СУБД и файловым хранилищем без перезапуска приложения и без модификации бизнес-логики.

После определения функциональных требований была спроектирована общая архитектура программы. Для обеспечения надежности и упрощения дальнейшей поддержки проекта было принято решение разделить систему на несколько независимых уровней в рамках многослойной архитектуры (Layered Architecture). В результате проектирования были выделены следующие модули:

- уровень представления (Presentation Layer): контроллеры, обрабатывающие HTTP-запросы и передающие данные в шаблоны Thymeleaf;
- уровень бизнес-логики (Service Layer): сервисы, инкапсулирующие правила обработки данных и оркестрирующие взаимодействие между портами;
- уровень портов (Port Layer): интерфейсы, определяющие минимально необходимый контракт доступа к данным;
- уровень адаптеров (Adapter Layer): конкретные реализации портов для СУБД (DbAdapter) и JSON (JsonAdapter);
- уровень хранения (Repository/Storage Layer): Spring Data JPA-репозитории и файловая система.

Такое разделение позволило изолировать логику обработки данных от способов их отображения и хранения, что значительно повысило чистоту архитектуры и обеспечило соблюдение принципа инверсии зависимостей (Dependency Inversion Principle) [3].

Для реализации механизма переключения был спроектирован класс DataModeService, использующий паттерн Стратегия (Strategy). Он хранит выбранный режим (DB или JSON) в HttpSession и предоставляет сервисам метод

					МИВУ.09.03.02-09.000	ПЗ	Лист
							12
Изм.	Лист	№ докум.	Подпись	Дата			

currentPort()), возвращающий нужный адаптер. Это обеспечило соблюдение принципа открытости/закрытости (Open/Closed Principle) из SOLID.

2.2 Проектирование информационной модели и базы данных

Важнейшим этапом проектирования системы является разработка информационной модели, которая должна обеспечивать целостность, непротиворечивость и минимальную избыточность хранимых данных. Поскольку разрабатываемый сервис предполагает работу со сложными динамическими структурами, в качестве основной модели хранения была выбрана реляционная модель [2].

На этапе проектирования была разработана логическая схема базы данных, включающая следующие основные сущности:

- сущность «Пользователь» (Users): базовая для обеспечения безопасности;
- сущность «Фильм» (Movies): центральный объект, содержащий метаданные и связи ManyToOne с режиссером и ManyToMany с жанрами и актерами;
- сущность «Сериал» (Series): содержит метаданные о многосерийном контенте;
- сущности «Сезон» (Seasons) и «Эпизод» (Episodes): реализуют иерархию со связями ON DELETE CASCADE, что гарантирует автоматическую очистку связанных структур при удалении сериала;
- справочные сущности: Genres, Actors, Directors.

При проектировании межтабличных связей особое внимание было уделено механизмам поддержания целостности. Спроектировано создание индексов по внешним ключам и полям, участвующим в частом поиске (например, по названию фильма или статусу), что обеспечивает высокую скорость работы системы при росте объема данных.

					МИВУ.09.03.02-09.000	ПЗ	Лист
							13
Изм.	Лист	№ докум.	Подпись	Дата			

В результате проектирования была получена нормализованная схема данных, которая позволяет эффективно хранить информацию. Финальная ER-диаграмма представлена на рисунке 2.

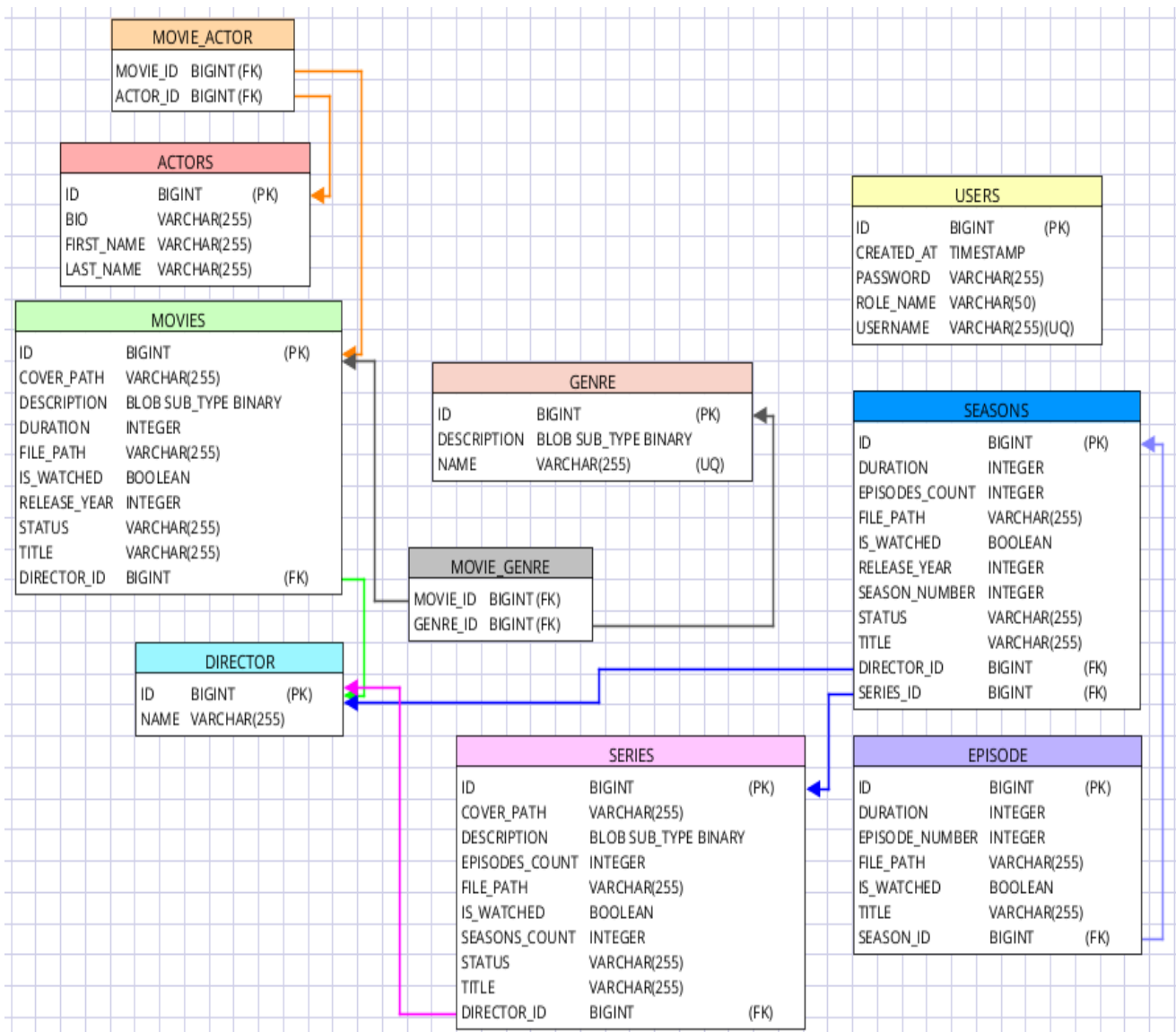


Рисунок 2 □ ER-диаграмма

3 Разработка приложения

3.1 Архитектурные особенности реализации

При проектировании и разработке приложения особое внимание было уделено соблюдению современных архитектурных принципов, что обеспечивает гибкость, масштабируемость и легкость поддержки программного продукта. В основе системы лежат несколько ключевых концепций.

Многослойная архитектура (Layered Architecture): приложение разделено на логические слои, взаимодействие между которыми строго регламентировано. Использование интерфейсов и сервисов позволяет изолировать бизнес-логику от деталей реализации базы данных или файловой системы.

Управление транзакциями: для обеспечения целостности данных при выполнении сложных операций (например, каскадное сохранение сериала, сезона и эпизодов) в слое сервисов применяется аннотация `@Transactional`. Это гарантирует, что операция будет выполнена либо полностью, либо не выполнена вовсе, предотвращая появление «частично сохраненных» данных.

Инверсия управления (IoC) и внедрение зависимостей (DI): использование механизмов Spring Framework позволило добиться слабой связанности (low coupling) между классами. Компоненты системы не создают зависимости вручную, а получают их через конструкторы или аннотацию `@Autowired`, что значительно упрощает модульное тестирование.

Безопасность как сквозная функциональность: архитектура безопасности интегрирована на уровне фильтров запросов. Благодаря Spring Security, проверка прав доступа осуществляется до того, как запрос попадает в контроллер, что позволяет централизованно управлять доступом.

3.2 Реализация информационной модели и базы данных

В полном соответствии с ER-диаграммой, разработанной на этапе проектирования, была реализована логическая схема базы данных на уровне программного кода.

					МИВУ.09.03.02-09.000	ПЗ	Лист
							15
Изм.	Лист	№ докум.	Подпись	Дата			

Структура БД реализована с помощью Entity-классов, аннотированных @Entity и @Table. Для описания связей использованы аннотации @ManyToOne, @OneToMany и @ManyToMany с указанием параметров каскадирования (CascadeType.ALL) и удаления осиротевших записей (orphanRemoval = true).

Доступ к данным унифицирован через интерфейсы репозитиев (MovieRepository, SeriesRepository и др.), которые наследуются от базового интерфейса JpaRepository. Это обеспечило прямое соответствие программных интерфейсов слою Persistence и позволило использовать встроенные методы CRUD без написания ручных SQL-запросов [4].

3.3 Реализация системы адаптеров хранилища (Port–Adapter)

Центральной особенностью разработанного сервиса является гибкая система настройки хранилища. В отличие от традиционных приложений, данное решение поддерживает два независимых режима работы, переключаемых в рамках пользовательской сессии.

Реализация опирается на полиморфную структуру данных. Для каждой сущности определен интерфейс-порт (например, MoviePort), описывающий методы findAll(), findById() и save().

Для режима БД реализованы классы DbAdapter, которые используют Spring Data JPA для взаимодействия с Firebird.

Для режима JSON реализованы классы JsonAdapter, которые используют библиотеку Jackson Databind для чтения и записи данных в файлы формата .json в директории /home/student/cinema-json/.

Ключевым техническим решением стала реализация паттерна Стратегия в сервисах. Метод currentPort() динамически возвращает нужный адаптер на основе значения, сохраненного в DataModeService.

Реализация динамики интерфейса представлена на рисунке 3.

```

@Service @Alius2005
public class MovieService {

    @Autowired @Qualifier("dbMoviePort")
    private MoviePort dbMovieAdapter;
    @Autowired @Qualifier("jsonMoviePort")
    private MoviePort jsonMovieAdapter;
    @Autowired
    private DataModeService dataModeService;

    private MoviePort currentPort() { 3 usages @Alius2005
        DataMode mode = dataModeService.getMode();
        if (mode == DataMode.JSON) {
            return jsonMovieAdapter;
        } else {
            return dbMovieAdapter;
        }
    }
}

```

Рисунок 3 □ Пример реаризации currentPort

Это решение обеспечивает соблюдение принципа единственной ответственности и позволяет легко добавить новое хранилище (например, MongoDB) в будущем без изменения кода сервисов.

3.4 Реализация управления жизненным циклом медиаконтента

Управление медиаконтентом реализовано через слой контроллеров (AdminController, AdminSeriesController), который обеспечивает полный цикл CRUD-операций. Важным аспектом является механизм разграничения прав доступа.

Система гарантирует, что операции редактирования или удаления доступны только пользователям с ролями ADMIN или moderator. Проверка осуществляется с помощью аннотации `@PreAuthorize("hasRole('MODERATOR') or`

					МИВУ.09.03.02-09.000	ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата			17

hasRole('ADMIN'))" на уровне методов контроллеров, а также через конфигурацию SecurityFilterChain.

Кроме того, реализована логика каскадного создания структуры сериала. При добавлении сериала через админ-панель метод saveSeriesWithStructure() автоматически создает запись сериала, связанный с ним сезон и заданное количество эпизодов, формируя пути к файлам по шаблону {folder}/{seasonNumber} сезон/{episodeNumber} серия.mp4.

3.5 Реализация интерфейса просмотра и воспроизведения

Процесс воспроизведения контента является наиболее ответственным этапом работы сервиса. При переходе по ссылке на фильм или эпизод система динамически формирует HTML-страницу, используя серверный шаблонизатор Thymeleaf.

Для воспроизведения видео используется стандартный HTML5-тег <video> с атрибутом controls. Раздача медиафайлов обеспечивается конфигурационным классом WebConfig, который регистрирует обработчик статических ресурсов, направляя запросы по шаблону /media/** в локальную директорию file:/home/student/videos/.

Для сериалов реализована полноценная навигация между эпизодами. Контроллер SeriesPlayerController подготавливает для шаблона данные о текущем, предыдущем и следующем эпизодах, а также список всех эпизодов сезона. В интерфейсе это отображается в виде кнопок навигации и визуальной панели с номерами серий, где текущий эпизод выделяется акцентным цветом через динамическое добавление CSS-класса .current-ep.

3.6 Реализация модуля рекомендаций и уведомлений

Для улучшения пользовательского опыта в системе предусмотрен модуль рекомендаций. Сервис CurationService отвечает за формирование персональной рекомендации «фильма дня» на главной странице. Его логика включает

					МИВУ.09.03.02-09.000	ПЗ	Лист
							18
Изм.	Лист	№ докум.	Подпись	Дата			

получение списка фильмов, присвоение каждому фильму тега настройки через компонент MoodAnalyzer и случайный выбор одного из них.

Ключевым техническим решением в этом модуле стала реализация системы уведомлений о новых релизах на основе паттерна Наблюдатель (Observer). Это позволило применить паттерн проектирования, обеспечивающий легкое расширение каналов уведомлений без изменения основного кода приложения.

Интерфейс ReleaseNotifier имеет три реализации:

- EmailNotifier (помечен как @Primary): имитация отправки email-уведомления;
- InAppNotifier: уведомление внутри приложения;
- MoodNotifier: персонализированное уведомление с учетом настройки пользователя.

Класс ReleaseNotifierService получает список всех зарегистрированных наблюдателей через механизм Dependency Injection (@Autowired List<ReleaseNotifier>) и обходит его, вызывая метод sendNotification() у каждого. Использование отдельных классов для каждого канала реализует принцип единственной ответственности, что упрощает отладку и поддержку кода.

					МИВУ.09.03.02-09.000	ПЗ	Лист
							19
Изм.	Лист	№ докум.	Подпись	Дата			

4 Тестирование

Одним из важнейших этапов разработки программного продукта является его тестирование и отладка. Тестирование позволяет выявить скрытые и явные дефекты программы, а также подтвердить её пригодность для практического применения. В рамках курсового проекта была применена комбинированная стратегия тестирования, включающая как ручную верификацию пользовательских сценариев, так и автоматизированное модульное и интеграционное тестирование.

Цель тестирования — проверить корректность работы всех ключевых подсистем приложения CinemaHome: механизма динамической смены хранилища (Port-Adapter), модуля рекомендаций, системы уведомлений (Observer), а также пользовательского интерфейса и ролевой модели безопасности.

4.1 Цели и стратегия тестирования

Стратегия тестирования была выстроена по принципу «от пользовательского опыта — к внутренним компонентам». Такой подход обусловлен спецификой разрабатываемого приложения: CinemaHome является пользовательским веб-сервисом, где именно поведение системы с точки зрения конечного пользователя (администратора, модератора или зрителя) является первичным критерием качества.

В связи с этим в проекте были выделены три уровня тестирования, применяемых в следующей последовательности:

Ручное (функциональное) тестирование — верификация пользовательских сценариев через браузер. Позволяет оценить систему целиком: от отрисовки HTML-шаблонов до взаимодействия с хранилищем данных и системой безопасности.

Автоматизированное интеграционное тестирование — проверка корректной загрузки Spring-контекста приложения и взаимодействия его компонентов.

					МИВУ.09.03.02-09.000	ПЗ	Лист
							20
Изм.	Лист	№ докум.	Подпись	Дата			

4.2 Ручное (функциональное) тестирование

Ручное функциональное тестирование проводилось в браузере Mozilla Firefox на локальном развёртывании приложения по адресу <http://localhost:8080>. Тестирование охватило все основные пользовательские сценарии: аутентификацию, работу с ролями, управление контентом, переключение режимов хранения, воспроизведение видео и работу модуля рекомендаций.

4.2.1 Сценарии функционального тестирования

Все проверенные сценарии сведены в таблицу 2.

Таблица 2 — Сценарии тестирования

№	Модуль	Действие	Ожидаемый результат	Статус
1.	Аутентификация	Регистрация нового пользователя	Учётная запись создана, редирект на /login?registered	Выполнено
2.	Аутентификация	Регистрация с неполными данными	HTML5-валидация блокирует отправку формы	Выполнено
3.	Аутентификация	Вход с корректными данными (admin/admin)	Установлена сессия, редирект на /	Выполнено
4.	Аутентификация	Вход с неверным паролем	Редирект на /login?error=true, отображение сообщения	Выполнено
5.	Роли	Доступ к /admin/movies без роли MODERATOR	HTTP 403, редирект на /access-denied	Выполнено
6.	Роли	Доступ к /admin/movies с ролью ADMIN	Отображение админ-панели	Выполнено
7.	Роли	Доступ к /admin/genres с ролью moderator	Отображение формы управления жанрами	Выполнено
8.	Фильмы (DB)	Добавление фильма через панель админа	Фильм сохранён в Firebird, отображается в списке	Выполнено
9.	Фильмы (JSON)	Добавление фильма в режиме JSON	Запись в movies.json	Выполнено
10.	Сериалы	Добавление сериала (1 сезон, 10 эпизодов)	Каскадно созданы Series + Season + 10 Episode	Выполнено
11.	Переключение режима	POST /mode с параметром mode=JSON	Режим сохранён в HttpSession, адаптеры переключены	Выполнено
12.	Плеер (фильм)	Открытие /player/movie/{id}	Отображение HTML5-плеера с src /media/...	Выполнено

Продолжение таблицы 7 — Сценарии тестирования

№	Модуль	Действие	Ожидаемый результат	Статус
13.	Аутентификация	Регистрация нового пользователя	Учётная запись создана, редирект на /login?registered	Выполнено
14.	Аутентификация	Регистрация с неполными данными	HTML5-валидация блокирует отправку формы	Выполнено
15.	Плеер (сериал)	Переход к следующей серии	Кнопка «Следующая серия» ведёт на следующий эпизод	Выполнено
16.	Плеер (сериал)	Переход к предыдущей серии	Кнопка «Предыдущая серия» ведёт на предыдущий эпизод	Выполнено
17.	Рекомендации	GET / с пустым каталогом	Отображается блок «Фильмов пока нет»	Выполнено
18.	Рекомендации	GET / с заполненным каталогом	Отображается случайный фильм с тегом настройки	Выполнено
19.	API-документация	GET /swagger-ui.html	Открыта страница Swagger UI	Выполнено
20.	Observer	Создание нового фильма через админ-панель	В консоли: 3 уведомления (Email, InApp, Mood)	Выполнено

4.2.2 Регистрация с некорректными данными

Для проверки надёжности системы валидации был проведён тест на регистрацию с заведомо неполными данными. Браузерная валидация HTML5-атрибута required и серверная валидация Spring заблокировали отправку формы. Результат представлен на рисунке 4.

4.2.3 Авторизация с некорректными данными

Была предпринята попытка входа под несуществующей учётной записью. Spring Security корректно обработал ситуацию: CustomUserDetailsService выбросил UsernameNotFoundException, и на странице /login?error=true было отображено красное сообщение «Неверный логин или пароль» через Thymeleaf-условие th:if=" \${param.error} ". Результат перехвата ошибки сервером показан на рисунке 5.

Регистрация

Имя пользователя

...

! Заполните это поле.

Создать аккаунт

Уже есть аккаунт? [Войти](#)

The registration form contains a text input field for the username, a dropdown menu, and a blue 'Создать аккаунт' button. A red error message 'Заполните это поле.' is displayed next to the dropdown menu. Below the form is a link 'Войти' for users who already have an account.

Рисунок 4 — Попытка регистрации с некорректными данными

Вход

Неверный логин или пароль

Имя пользователя

Пароль

Войти

Нет аккаунта? [Зарегистрироваться](#)

The login form features a red error message 'Неверный логин или пароль' at the top. Below it are input fields for the username and password, followed by a blue 'Войти' button. At the bottom, there is a link 'Зарегистрироваться' for users who do not have an account.

Рисунок 5 — Попытка авторизации с некорректными данными

4.2.4 Тестирование ролевой модели и страницы 403

Попытка доступа к админ-панели (/admin/movies) пользователем с ролью user блокируется фильтром Spring Security с HTTP-кодом 403 Forbidden и перенаправлением на страницу /access-denied. Это обеспечивается конфигурацией `exceptionHandling(ex -> ex.accessDeniedPage("/access-denied"))` в классе `SecurityConfig` и отдельным контроллером `AccessDeniedController`. Авторизованный пользователь с ролью ADMIN или MODERATOR успешно получает доступ к формам добавления контента. Результаты представлены на рисунках 6 и 7

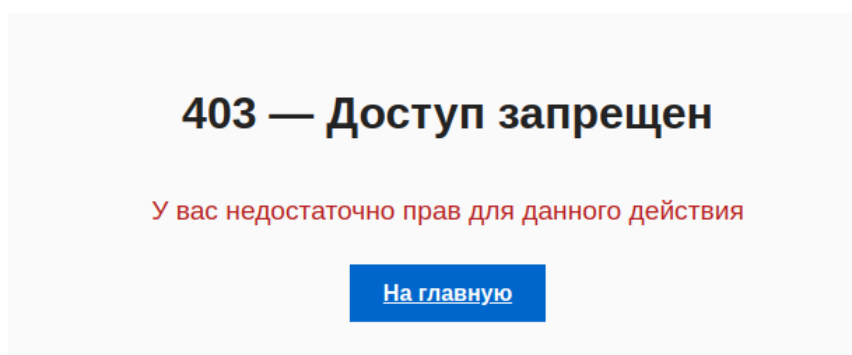


Рисунок 6 — Отказ в доступе (HTTP 403) с редиректом на /access-denied

4.2.5 Переключение режима хранения (DB <—> JSON)

Ключевой сценарий проекта — динамическая смена хранилища. При переходе на страницу /mode и выборе опции «JSON файлы» с последующим POST-запросом, `DataModeController` сохраняет значение `DataMode.JSON` в `HttpSession`. Все последующие обращения к сервисам (`MovieService`, `SeriesService`, `DirectorService` и др.) через метод `currentPort()` возвращают соответствующие `Json*Adapter`. Добавленный в этом режиме фильм сохраняется в файл /home/student/cinema-json/movies.json. Результат представлен на рисунке 8.

					МИВУ.09.03.02-09.000	ПЗ	Лист
							24
Изм.	Лист	№ докум.	Подпись	Дата			

Панель админа

Главная

Жанры

Фильмы

Актеры

Сериалы

Режиссеры

Добавить фильм

Название

Описание

/home/student/videos/ваше кино.mp4

Настроение (Romantic, Calm...)

Сохранить

Рисунок 7 — Успешный доступ к админ-панели

localhost:8080/mode

Выберите источник данных

☒ База данных (Firebird)

☐ JSON файлы

Продолжить

Рисунок 8 — Страница переключения режима хранения

4.2.6 Воспроизведение видео через HTML5-плеер

При переходе на `/player/movie/{id}` контроллер `PlayerController` загружает объект `MovieDto` и передаёт его в шаблон `player/movie-player.html`. Thymeleaf формирует HTML5-тег `<video>` с источником `/media/{filePath}`, а `WebConfig` раздаёт файл из директории `file:/home/student/videos/`. Результат представлен на рисунке 9.

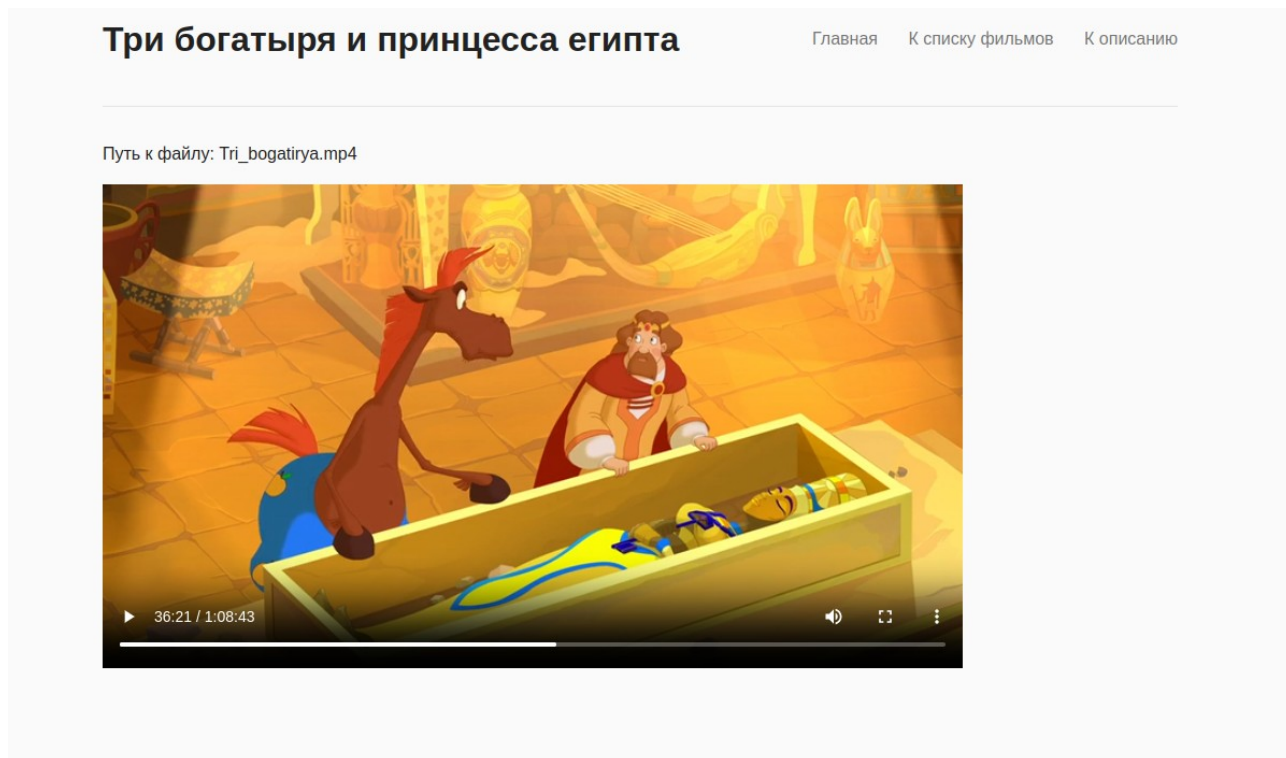


Рисунок 9 — Встроенный HTML5-видеоплеер

4.2.7 Навигация по эпизодам сериала

При просмотре эпизода сериала через `/player/series/{episodeId}` пользователю доступны:

- кнопка «Предыдущая серия» (скрывается для первого эпизода);
- кнопка «Следующая серия» (скрывается для последнего эпизода);
- визуальная панель со всеми эпизодами текущего сезона, где текущий эпизод выделен акцентным цветом (CSS-класс `.current-ep`).

Данная логика реализуется через методы `findPrevious()` и `findNext()` в `EpisodePort` и поддерживается обоими адаптерами (DB через

findBySeason_IdAndEpisodeNumber, JSON через Stream API с фильтрацией).
Результат представлен на рисунке 10.

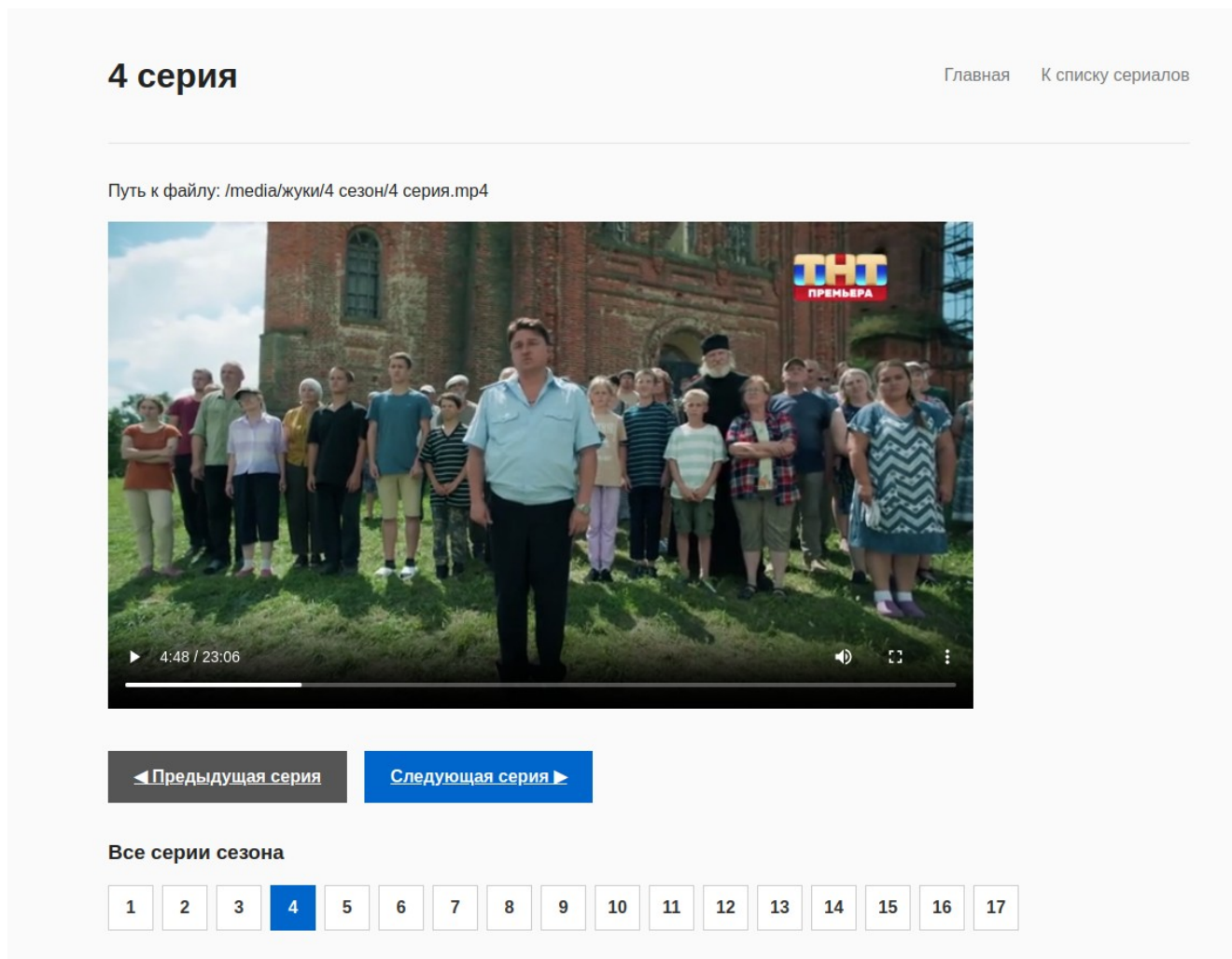


Рисунок 10 — Плеер сериала с навигацией по эпизодам

4.2.8 Панель управления контентом

На главной странице присутствует блок быстрого перехода к админ-формам с выпадающим списком из 5 пунктов: Фильм, Сериал, Жанр, Актёр, Режиссёр. JavaScript-обработчик `goToAddPage()` использует конструкцию `switch` для навигации к соответствующему URL (`/admin/movies`, `/admin/series`, `/admin/genres`, `/admin/actors`). Результат представлен на рисунке 15.

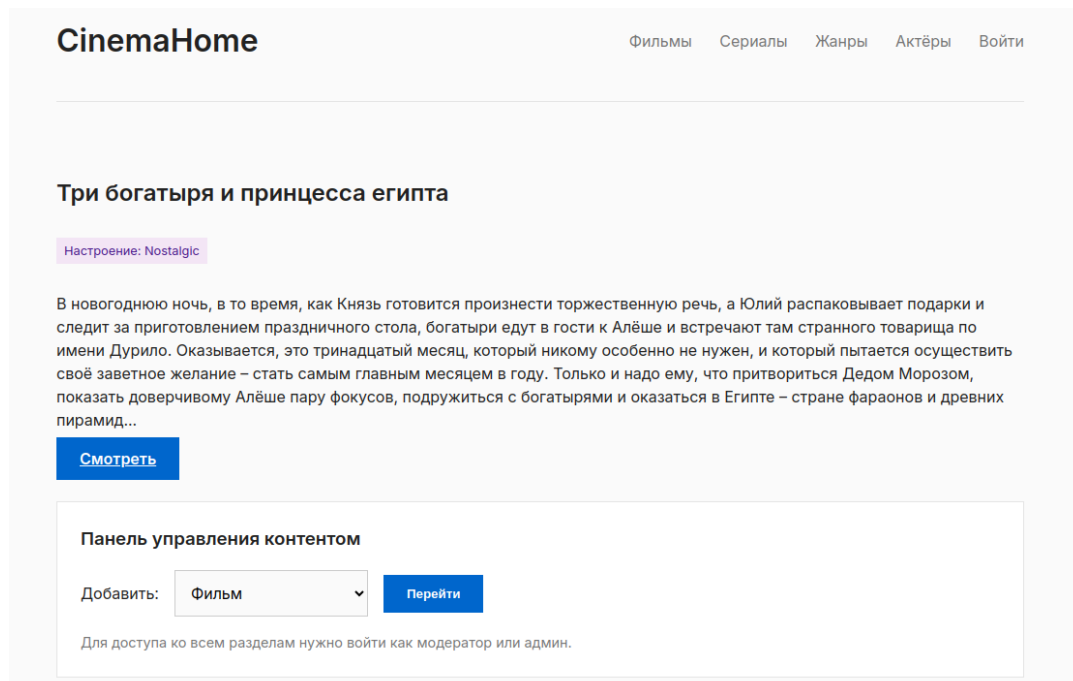


Рисунок 11 — Панель управления контентом на главной странице

4.2.9 API-документация Swagger UI

Библиотека `springdoc-openapi-starter-webmvc-ui` версии 2.3.0 автоматически генерирует интерактивную документацию по REST API. При переходе на `/swagger-ui.html` открывается страница с полным описанием эндпоинтов (RecommendationController, публичные ресурсы). Результат представлен на рисунке 12.

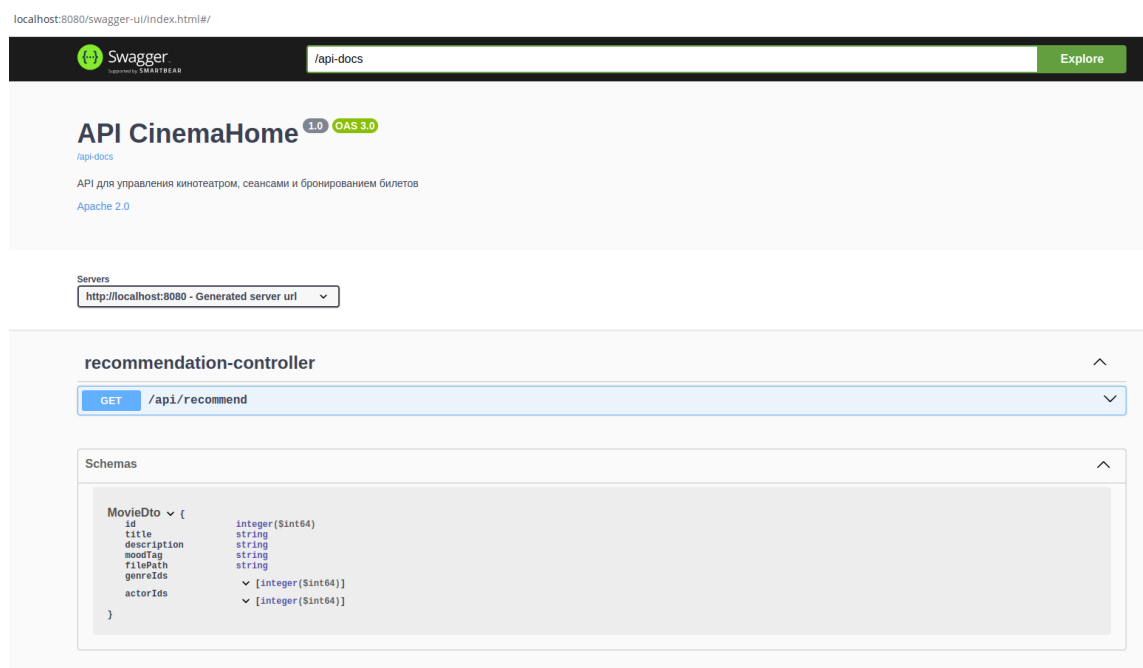


Рисунок 12 — Swagger UI

4.3 Инструментарий автоматизированного тестирования

После подтверждения работоспособности всех пользовательских сценариев через ручное тестирование, были написаны автоматизированные тесты для наиболее критичных с точки зрения бизнес-логики модулей. Для их реализации был выбран следующий набор библиотек, подключённых через build.gradle. Каждая библиотека решает свою специфическую задачу и работает в связке с остальными.

Инструментарий использованных автоматизированных тестов представлен на таблице 3.

Таблица 3 — Инструментарий автоматизированных тестов

Инструмент	Версия	Назначение
JUnit 5 (Jupiter)	5.x (через spring-boot-starter-test)	Основной фреймворк для написания и запуска тестов
Mockito	5.+	Создание mock-объектов для изоляции тестируемых компонентов
AssertJ	3.x (через spring-boot-starter-test)	Fluent-стиль утверждений для повышения читаемости тестов
Spring Boot Test	3.4.0	Интеграционные тесты с загрузкой полного контекста
ReflectionTestUtils	Spring Test	Утилита для инъекции приватных полей в тестируемые объекты
Gradle Test Task	8.5	Автоматический прогон тестов и генерация HTML-отчётов

4.3.1 JUnit 5 (Jupiter) — основа тестового фреймворка

JUnit 5 — современный стандарт модульного тестирования в Java-экосистеме, пришедший на смену JUnit 4. В отличие от предшественника, JUnit 5 имеет модульную архитектуру, состоящую из трёх подпроектов:

1. JUnit Platform — платформа для запуска тестов, предоставляющая API для интеграции с IDE и системами сборки;
2. JUnit Jupiter — новая программная модель для написания тестов с использованием аннотаций `@Test`, `@BeforeEach`, `@AfterEach`, `@ParameterizedTest`;

3. JUnit Vintage — движок для обратной совместимости с тестами JUnit 3 и JUnit 4 (в проекте не используется).

Принцип работы: каждый метод, помеченный аннотацией `@Test`, рассматривается платформой как отдельный тестовый сценарий. JUnit Jupiter выполняет тесты в изолированных экземплярах тестового класса (по умолчанию создаётся новый экземпляр на каждый тест), что исключает побочные эффекты между тестами.

Аннотация `@BeforeEach` используется для подготовки тестового окружения перед каждым тестом (создание mock-объектов, инъекция зависимостей) с примеров на рисунке 13:

```
22
23     @BeforeEach  @ Alius2005
24     void setUp() {
25         movieRepository = Mockito.mock(MovieRepository.class);
26         moodAnalyzer = Mockito.mock(MoodAnalyzer.class);
27         curationService = new CurationService();
28
29         // подменяем приватные поля через рефлексию
30         injectField(curationService, fieldName: "movieRepository", movieRepository);
31         injectField(curationService, fieldName: "moodAnalyzer", moodAnalyzer);
32     }
```

Рисунок 13 — Пример использования `@BeforeEach`

JUnit 5 подключается в проект транзитивно через стартер `spring-boot-starter-test` и активируется в Gradle директивой `test { useJUnitPlatform() }`.

4.3.2 Mockito — мокирование зависимостей

Mockito — самый популярный фреймворк для создания mock-объектов (подставных объектов) в Java. Mock-объекты имитируют поведение реальных зависимостей (репозиторий, сервисов, внешних API) без обращения к ним, что позволяет тестировать классы изолированно.

Принцип работы: Mockito использует динамическую генерацию байт-кода (через библиотеку ByteBuddy) для создания прокси-классов, реализующих нужный интерфейс или наследующих нужный класс. Поведение mock-объекта настраивается через fluent API пример на рисунке 14

					МИВУ.09.03.02-09.000	ПЗ	Лист
							30
Изм.	Лист	№ докум.	Подпись	Дата			

```

@Test  @Alius2005 *
void sendNotification_callsMoodAnalyzer() {
    // Создание mock-объекта
    MoodAnalyzer analyzer = mock(MoodAnalyzer.class);
    // Настройка поведения: при вызове analyzeMood() вернуть "Calm"
    when(analyzer.analyzeMood()).thenReturn(value: "Calm");

    MoodNotifier notifier = new MoodNotifier();
    injectField(notifier, fieldName: "moodAnalyzer", analyzer);

    notifier.sendNotification();
    // Проверка: был ли метод analyzeMood() вызван ровно 1 раз
    verify(analyzer, times(wantedNumberOfInvocations: 1)).analyzeMood();
}

```

Рисунок 14 – Пример использования Mockito

Ключевые возможности, использованные в проекте:

- mock(Class) — создание mock-объекта для заданного класса или интерфейса;
- when(...).thenReturn(...) — настройка возвращаемого значения при вызове метода;
- verify(..., times(N)) — проверка количества вызовов метода (важно для тестирования паттерна Observer).

Применение в проекте: Mockito используется во всех unit-тестах (MoodNotifierTest, CurationServiceTest) для изоляции тестируемых компонентов от их зависимостей. Например, в CurationServiceTest создаются mock-объекты MovieRepository и MoodAnalyzer, что позволяет протестировать логику сервиса без обращения к реальной базе данных или генератору случайных чисел.

Особенность конфигурации: в build.gradle Mockito подключён как testImplementation 'org.mockito:mockito-core:5.+', что обеспечивает использование актуальной версии 5.x с поддержкой inline-mock-maker (возможность мокать final-классы и статические методы).

4.3.3 AssertJ — читаемые утверждения

AssertJ — библиотека для написания утверждений (assertions) в стиле fluent API. В отличие от стандартных методов JUnit (assertEquals), AssertJ предлагает более выразительный синтаксис, который читается как английский текст.

Принцип работы: все утверждения строятся через фабричный метод `assertThat(actual)`, который возвращает объект-обёртку с цепочкой методов проверки пример представлен на рисунке 15.

```
14 class JsonMovieRepositoryTest { @ Alius2005
15
16     @Test @ Alius2005
17     void saveAndFindById_workWithTempFile() throws Exception {
18         // временный файл вместо /home/student/cinema-json/movies.json
19         File tmp = Files.createTempFile( prefix: "movies-test", suffix: ".json").toFile();
20         tmp.deleteOnExit();
21
22         JsonMovieAdapter adapter = new JsonMovieAdapter();
23
24         // подменяем приватную константу MOVIES_JSON_PATH
25         ReflectionTestUtils.setField(adapter, name: "MOVIES_JSON_PATH", tmp.getAbsolutePath());
26
27         MovieDto dto = new MovieDto();
28         dto.setTitle("Test movie");
29         dto.setDescription("Desc");
30         dto.setMoodTag("CaLm");
31
32         adapter.save(dto);
33
34         List<MovieDto> all = adapter.findAll();
35         assertThat(all).hasSize( expected: 1);
36
37         MovieDto saved = all.get(0);
38         assertThat(saved.getId()).isNotNull();
39         assertThat(saved.getTitle()).isEqualTo( expected: "Test movie");
40
41         MovieDto byId = adapter.findById(saved.getId());
42         assertThat(byId).isNotNull();
43         assertThat(byId.getTitle()).isEqualTo( expected: "Test movie");
44     }
45 }
```

Рисунок 15 – Пример использования AssertJ

Ключевые возможности, использованные в проекте:

- `hasSize(N)` — проверка размера коллекции;
- `isEqualTo(...)` — проверка равенства значений;

- isNotNull() / isNull() — проверка на null;
- специализированные утверждения для коллекций, строк, дат, исключений.

Применение в проекте: AssertJ используется во всех unit-тестах как основной инструмент проверки результатов. Подключается транзитивно через spring-boot-starter-test.

4.3.4 Spring Boot Test — интеграционное тестирование

Spring Boot Test — модуль фреймворка Spring, предоставляющий специализированные аннотации и утилиты для тестирования Spring-приложений. Ключевой аннотацией является @SpringBootTest, которая поднимает полный Spring-контекст приложения, включая все бины, конфигурации, сервисы и репозитории.

Принцип работы: при запуске теста с @SpringBootTest Spring Boot выполняет те же шаги инициализации, что и при старте реального приложения:

- сканирование классов с аннотациями @Component, @Service, @Repository, @Controller, @Configuration;
- создание бинов и внедрение зависимостей;
- инициализация DataSource, EntityManagerFactory, SecurityFilterChain;
- запуск CommandLineRunner (в проекте — AdminInitializer).

Если на любом из этих этапов возникает ошибка (например, конфликт бинов, отсутствующая зависимость), тест завершается с ошибкой, что позволяет обнаруживать проблемы конфигурации на ранней стадии.

4.3.5 ReflectionTestUtils — инъекция приватных полей

ReflectionTestUtils — утилита из модуля spring-test, предоставляющая методы для работы с приватными полями и методами через рефлексиию. Она является стандартом де-факто для инъекции зависимостей в unit-тестах, где не используется Spring-контекст.

Принцип работы: В unit-тестах мы не поднимаем Spring-контекст (это медленно и избыточно), а создаём тестируемые объекты через new. Однако эти

					МИВУ.09.03.02-09.000	ПЗ	Лист
							33
Изм.	Лист	№ докум.	Подпись	Дата			

объекты часто имеют приватные поля, помеченные @Autowired, которые в production-режиме заполняются контейнером Spring.

4.3.6 Gradle Test Task — автоматизация запуска тестов

Gradle Test Task — встроенная задача системы сборки Gradle, автоматически обнаруживающая тестовые классы и выполняющая их. Результатом работы является HTML-отчёт, доступный по пути build/reports/tests/test/index.html.

Принцип работы:

- Gradle сканирует директорию src/test/java/ на наличие классов с методами @Test;
- для каждого найденного класса создаётся отдельный тестовый runner;
- выполняются все тестовые методы, результаты (успех/падение/время) фиксируются;
- генерируется HTML-отчёт с детализацией по пакетам, классам и методам.

					МИВУ.09.03.02-09.000	ПЗ	Лист
							34
Изм.	Лист	№ докум.	Подпись	Дата			

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта было разработано веб-приложение CinemaHome — адаптивная платформа локального медиаконтента с динамической сменой хранилища. Основной целью проекта являлось создание многофункционального приложения, способного гибко управлять каталогом фильмов и сериалов, воспроизводить видеофайлы через встроенный HTML5-плеер и прозрачно переключаться между двумя независимыми хранилищами данных — реляционной СУБД Firebird и файловым хранилищем в формате JSON.

В процессе разработки была спроектирована гексагональная архитектура (Port-Adapter) с элементами Layered Architecture, что обеспечило чёткое разделение ответственности между слоями Controller, Service, Port, Adapter и Repository. Такое решение позволило добиться слабой связанности модулей и соответствия принципам SOLID, в частности Dependency Inversion Principle и Open/Closed Principle.

В рамках работы были реализованы:

- поддержка семи типов сущностей (Movie, Series, Season, Episode, Actor, Genre, Director, User);
- два параллельных набора адаптеров (Db*Adapter и Json*Adapter) для каждого порта;
- паттерн Strategy для динамического выбора хранилища через DataModeService;
- паттерн Observer для системы уведомлений о релизах (EmailNotifier, InAppNotifier, MoodNotifier);
- ролевая модель безопасности (ADMIN, moderator, user) на базе Spring Security и BCrypt;
- пользовательский интерфейс на Thymeleaf с минималистичной CSS-вёрсткой;
- модуль рекомендаций (CurationService + MoodAnalyzer);

					МИВУ.09.03.02-09.000	ПЗ	Лист
							35
Изм.	Лист	№ докум.	Подпись	Дата			

- автоматическое создание администратора и модератора при старте (AdminInitializer);

- API-документация через springdoc-openapi (Swagger UI).

В качестве технологического стека были выбраны Java 21, Spring Boot 3.4, Hibernate 6.6.2 с Community Dialects для Firebird, Jackson Databind 2.17.0, Thymeleaf, Spring Security, Lombok 1.18.36, сборщик Gradle 8.5.

Корректность работы ключевых модулей подтверждена модульными тестами (JUnit 5 + Mockito + AssertJ). Все 5 тестов проходят успешно (success rate 100 %).

В результате выполнения курсового проекта была создана полнофункциональная и масштабируемая платформа локального медиаконтента, соответствующая всем требованиям технического задания. Разработанный продукт наглядно демонстрирует применение принципов объектно-ориентированного проектирования, современных архитектурных паттернов и промышленных стандартов веб-разработки на платформе Java.

					МИВУ.09.03.02-09.000	ПЗ	Лист
							36
Изм.	Лист	№ докум.	Подпись	Дата			

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 1 Ермаков, А. В. Разработка приложений на языке JAVA : учебное пособие / А. В. Ермаков, М. С. Королёв, А. К. Кузьмин. — Саратов : Саратовский государственный технический университет имени Ю.А. Гагарина, ЭБС АСВ, 2024. — 128 с. — ISBN 978-5-7433-3635-7.

2 2 Лазицкас, Е. А. Базы данных и системы управления базами данных [Электронный ресурс] : учебное пособие / Е. А. Лазицкас, И. Н. Загумённикова, П. Г. Гилевский. — Минск : РИПО, 2018. — 268 с. — Режим доступа: <http://www.iprbookshop.ru/93382.html>.

3 3 Мирошников, А. И. Архитектура систем управления базами данных [Электронный ресурс] : учебное пособие / А. И. Мирошников. — Липецк : ЛГТУ, ЭБС АСВ, 2018. — 94 с. — Режим доступа: <http://www.iprbookshop.ru/83189.html>.

4 4 Суханов, В. И. Разработка веб-приложений на платформе Spring : учебно-методическое пособие / В. И. Суханов ; под ред. С. И. Тимошенко. — Екатеринбург : Изд-во Уральского ун-та, 2023. — 180 с. — Режим доступа: <https://www.iprbookshop.ru/157380.html>.

5 5 Официальная документация Spring Boot 3.4 [Электронный ресурс]. — URL: <https://docs.spring.io/spring-boot/docs/3.4.0/reference/html/>